МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

Отчет о выполнении практической работы № 4

по дисциплине ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ
на тему «ПЕРЕГРУЗКА ОПЕРАТОРОВ И ПРОТОКОЛЫ ОБЪЕКТОВ В PYTHON»
Вариант № 19

Выполнил студент: Поваров Тимур Алексеевич
Группа: ТКБО-01-25

Москва, 2026

🔗 Ссылка на блокнот

https://colab.research.google.com/drive/1DCWd4apC-AWYqvH-29-Qu3dM4JF6x7Al?usp=sharing

🔗 Ссылка на DeepSeek

Ваша ссылка

🎯 ЦЕЛЬ РАБОТЫ

1. Закрепить понимание принципов объектно-ориентированного программирования в Python.
2. Научиться использовать магические методы для:

- перегрузки арифметических операторов;
- сравнения объектов пользовательских классов;
- вычисления хэша объектов;
- определения логической истинности объектов;
- работы с объектами как с коллекциями (через индексацию);
- создания и использования итерируемых объектов и итераторов.

3. Сформировать навыки проектирования классов с корректным поведением в вы-ражениях, условиях, контейнерах и циклах.

🏁🏁🏁🏁🏁🏁🏁🏁 Задание 1 🏁🏁🏁🏁🏁🏁🏁🏁

📝 Задание 1. Перегрузка арифметических операторов и операций сравнения

🎯 Формулировка задания 1:

* 🧬 «Трекер длины геномных последовательностей»

📌 Суть задания Разработать программу для работы с набором измерений длины геномных последовательностей, где каждое измерение хранится в парах оснований (п.н.). Программа должна управлять коллекцией измерений, выполнять операции с длинами, сортировку и анализ.

Реализовать классы:

SequenceLength — одно измерение длины последовательности в п.н.; GenomeLog — коллекция измерений длины последовательностей. 📋 Требования к SequenceLength Хранит длину в парах оснований (100..1000000 п.н.); Магические методы: **add**, **sub** — сложение/вычитание с числами и другими объектами; **iadd**, **isub** — изменение длины «на месте»; **eq**, **lt**, **le** — сравнение длин последовательностей; **str** — вывод в формате X п.н. (Y кп.н.). Поддерживает: объект ± число; объект ± объект. 📋 Требования к GenomeLog Хранит список объектов SequenceLength; Магические методы: **getitem**, **setitem**, **len**, **iter**; Методы: sort() —

сортировка по длине; longest() — самая длинная последовательность; shortest() — самая короткая последовательность; average_length() — средняя длина последовательностей. 👮 Ввод данных Выберите способ ввода: 1 — Ввести вручную 2 — Случайная генерация 1 → пользователь вводит количество последовательностей и их длины; 2 → программа генерирует N случайных значений (1000..500000 п.н.). 🧪 Операции в программе Показать все измерения длины последовательностей Отсортировать по длине Найти самую короткую и самую длинную последовательность Посчитать среднюю длину последовательностей Скорректировать все значения на X п.н. (коррекция данных)*

Описание классов

```
import random

class SequenceLength:
    def __init__(self, length):
        if not isinstance(length, int):
            raise TypeError
        if length < 100 or length > 1000000:
            raise ValueError
        self.length = length

    def _validate(self, value):
        if value < 100 or value > 1000000:
            raise ValueError
        return value

    def __add__(self, other):
        if isinstance(other, SequenceLength):
            value = self.length + other.length
        elif isinstance(other, int):
            value = self.length + other
        else:
            raise TypeError
        return SequenceLength(self._validate(value))

    def __sub__(self, other):
        if isinstance(other, SequenceLength):
            value = self.length - other.length
        elif isinstance(other, int):
            value = self.length - other
        else:
            raise TypeError
        return SequenceLength(self._validate(value))

    def __iadd__(self, other):
        if isinstance(other, SequenceLength):
            value = self.length + other.length
        elif isinstance(other, int):
            value = self.length + other
        else:
            raise TypeError
        self.length = self._validate(value)
        return self

    def __isub__(self, other):
        if isinstance(other, SequenceLength):
            value = self.length - other.length
        elif isinstance(other, int):
            value = self.length - other
        else:
            raise TypeError
        self.length = self._validate(value)
        return self

    def __eq__(self, other):
        return self.length == other.length

    def __lt__(self, other):
        return self.length < other.length

    def __le__(self, other):
        return self.length <= other.length

    def __str__(self):
```

```
        kb = self.length / 1000
        return f"{self.length} п.н. ({kb:.2f} кп.н.)"


class GenomeLog:
    def __init__(self, data=None):
        self.data = data if data else []

    def __getitem__(self, index):
        return self.data[index]

    def __setitem__(self, index, value):
        if not isinstance(value, SequenceLength):
            raise TypeError
        self.data[index] = value

    def __len__(self):
        return len(self.data)

    def __iter__(self):
        return iter(self.data)

    def sort(self):
        self.data.sort()

    def longest(self):
        if not self.data:
            return None
        return max(self.data)

    def shortest(self):
        if not self.data:
            return None
        return min(self.data)

    def average_length(self):
        if not self.data:
            return 0
        total = sum(x.length for x in self.data)
        return total / len(self.data)

    def correct_all(self, value):
        for i in range(len(self.data)):
            self.data[i] += value
```

Функции создания и работы с экземплярами классов

```
def create_log():
    choice = input("1 — вручную, 2 — случайно: ")

    if choice == "1":
        n = int(input("Количество последовательностей: "))
        data = []

        for _ in range(n):
            while True:
                try:
                    length = int(input("Длина: "))
                    data.append(SequenceLength(length))
                    break
                except:
                    print("Ошибка ввода")

        return GenomeLog(data)

    if choice == "2":
        n = int(input("Количество последовательностей: "))
        data = []

        for _ in range(n):
            length = random.randint(1000, 500000)
            data.append(SequenceLength(length))

        return GenomeLog(data)
```

```
def show_all(log):
    if len(log) == 0:
        print("Нет данных")
        return

    for item in log:
        print(item)


def menu():
    log = create_log()

    while True:
        print()
        print("1 Показать все")
        print("2 Сортировать")
        print("3 Самая короткая и длинная")
        print("4 Средняя длина")
        print("5 Корректировать на X")
        print("0 Выход")

        cmd = input("Выбор: ")

        if cmd == "1":
            show_all(log)

        elif cmd == "2":
            log.sort()
            print("Отсортировано")

        elif cmd == "3":
            print("Короткая:", log.shortest())
            print("Длинная:", log.longest())

        elif cmd == "4":
            print("Средняя:", log.average_length())

        elif cmd == "5":
            x = int(input("Коррекция X: "))
            log.correct_all(x)

        elif cmd == "0":
            break


menu()
```

Основная программа

```
import random

class SequenceLength:
    def __init__(self, length):
        if not isinstance(length, int):
            raise TypeError
        if length < 100 or length > 1000000:
            raise ValueError
        self.length = length

    def __add__(self, other):
        if isinstance(other, SequenceLength):
            return SequenceLength(self.length + other.length)
        if isinstance(other, int):
            return SequenceLength(self.length + other)
        raise TypeError

    def __sub__(self, other):
        if isinstance(other, SequenceLength):
            return SequenceLength(self.length - other.length)
        if isinstance(other, int):
            return SequenceLength(self.length - other)
        raise TypeError
```

```
    def __iadd__(self, other):
        if isinstance(other, SequenceLength):
            self.length += other.length
        elif isinstance(other, int):
            self.length += other
        else:
            raise TypeError
        return self

    def __isub__(self, other):
        if isinstance(other, SequenceLength):
            self.length -= other.length
        elif isinstance(other, int):
            self.length -= other
        else:
            raise TypeError
        return self

    def __eq__(self, other):
        return self.length == other.length

    def __lt__(self, other):
        return self.length < other.length

    def __le__(self, other):
        return self.length <= other.length

    def __str__(self):
        kb = self.length / 1000
        return f"{self.length} п.н. ({kb:.2f} кп.н.)"


class GenomeLog:
    def __init__(self, data=None):
        self.data = data if data else []

    def __getitem__(self, index):
        return self.data[index]

    def __setitem__(self, index, value):
        if not isinstance(value, SequenceLength):
            raise TypeError
        self.data[index] = value

    def __len__(self):
        return len(self.data)

    def __iter__(self):
        return iter(self.data)

    def sort(self):
        self.data.sort()

    def longest(self):
        return max(self.data)

    def shortest(self):
        return min(self.data)

    def average_length(self):
        if not self.data:
            return 0
        total = sum(x.length for x in self.data)
        return total / len(self.data)

    def correct_all(self, value):
        for i in range(len(self.data)):
            self.data[i] += value


def create_log():
    choice = input("1 — вручную, 2 — случайно: ")

    if choice == "1":
        n = int(input("Количество последовательностей: "))
        data = []
        for _ in range(n):
```

```
            length = int(input("Длина: "))
            data.append(SequenceLength(length))
        return GenomeLog(data)

    if choice == "2":
        n = int(input("Количество последовательностей: "))
        data = []
        for _ in range(n):
            length = random.randint(1000, 500000)
            data.append(SequenceLength(length))
        return GenomeLog(data)


def show_all(log):
    for item in log:
        print(item)


def menu():
    log = create_log()

    while True:
        print()
        print("1 Показать все")
        print("2 Сортировать")
        print("3 Самая короткая и длинная")
        print("4 Средняя длина")
        print("5 Корректировать на X")
        print("0 Выход")

        cmd = input("Выбор: ")

        if cmd == "1":
            show_all(log)

        elif cmd == "2":
            log.sort()
            print("Отсортировано")

        elif cmd == "3":
            print("Короткая:", log.shortest())
            print("Длинная:", log.longest())

        elif cmd == "4":
            print("Средняя:", log.average_length())

        elif cmd == "5":
            x = int(input("Коррекция X: "))
            log.correct_all(x)

        elif cmd == "0":
            break


menu()
```

Задание 2

📝 Задание 2. Сравнение, хэш и логическая истинность объектов

🎯 Формулировка задания 2:

* 🧬 «Коллекция генетических образцов»

📌 Суть задания Разработать программу для работы с множеством генетических образцов в 3D-пространстве (GC_состав, температура_плавления, длина_фрагмента). Программа должна хранить коллекцию образцов, выполнять операции с параметрами, фильтровать образцы по GC-составу и искать уникальные образцы.

Реализовать:

GeneticSample — один генетический образец; SampleCollection — коллекция генетических образцов. 📋 Требования к GeneticSample Хранит параметры: GC-состав (20..80%), температура плавления (50..100°C), длина фрагмента (100..10000 п.н.);

Магические методы: **add**, **sub** — смещение параметров; **mul** — масштабирование параметров; **eq**, **hash** — для поиска уникальных образцов; **bool** — True, если GC-состав > 50% (GC-богатый), иначе False; **str** — вывод (GC: X%, Tm: Y°C, длина: Z п.н.). 📋 Требования к SampleCollection Хранит список объектов GeneticSample; Магические методы: **getitem**, **len**, **iter**; Методы: remove_gc_poor() — удалить GC-бедные образцы (≤50%); unique_samples() — оставить только уникальные образцы; longest_fragment() — образец с самым длинным фрагментом; collection_center() — средние параметры коллекции. 👷 Ввод данных 1 — Ввести параметры вручную 2 — Сгенерировать случайно При генерации: GC-состав [25, 75], температура [60, 95], длина [500, 5000]. 🧪 Операции в программе Показать все генетические образцы Удалить GC-бедные образцы Оставить только уникальные образцы Найти образец с самым длинным фрагментом Найти средние параметры коллекции Скорректировать все образцы на вектор*

Описание классов

```
import random

class GeneticSample:
    def __init__(self, gc, tm, length):
        if not (20 <= gc <= 80):
            raise ValueError
        if not (50 <= tm <= 100):
            raise ValueError
        if not (100 <= length <= 10000):
            raise ValueError

        self.gc = gc
        self.tm = tm
        self.length = length

    def __add__(self, other):
        if isinstance(other, GeneticSample):
            return GeneticSample(
                self.gc + other.gc,
                self.tm + other.tm,
                self.length + other.length
            )
        elif isinstance(other, (tuple, list)) and len(other) == 3:
            return GeneticSample(
                self.gc + other[0],
                self.tm + other[1],
                self.length + other[2]
            )
        else:
            raise TypeError

    def __sub__(self, other):
        if isinstance(other, GeneticSample):
            return GeneticSample(
                self.gc - other.gc,
                self.tm - other.tm,
                self.length - other.length
            )
        elif isinstance(other, (tuple, list)) and len(other) == 3:
            return GeneticSample(
                self.gc - other[0],
                self.tm - other[1],
                self.length - other[2]
            )
        else:
            raise TypeError

    def __mul__(self, k):
        if not isinstance(k, (int, float)):
            raise TypeError
        return GeneticSample(
            int(self.gc * k),
            int(self.tm * k),
            int(self.length * k)
        )

    def __eq__(self, other):
        return (self.gc, self.tm, self.length) == (other.gc, other.tm, other.length)
```

```
    def __hash__(self):
        return hash((self.gc, self.tm, self.length))

    def __bool__(self):
        return self.gc > 50

    def __str__(self):
        return f"GC: {self.gc}%, Tm: {self.tm}°C, Len: {self.length}"


class SampleCollection:
    def __init__(self, samples=None):
        self.samples = samples if samples else []

    def __getitem__(self, index):
        return self.samples[index]

    def __len__(self):
        return len(self.samples)

    def __iter__(self):
        return iter(self.samples)

    def remove_gc_poor(self):
        self.samples = [s for s in self.samples if s.gc > 50]

    def unique_samples(self):
        self.samples = list(set(self.samples))

    def longest_fragment(self):
        if not self.samples:
            return None
        return max(self.samples, key=lambda x: x.length)

    def collection_center(self):
        if not self.samples:
            return (0, 0, 0)

        gc = sum(s.gc for s in self.samples) / len(self.samples)
        tm = sum(s.tm for s in self.samples) / len(self.samples)
        length = sum(s.length for s in self.samples) / len(self.samples)

        return (gc, tm, length)

    def apply_vector(self, vector):
        new = []
        for s in self.samples:
            new.append(s + vector)
        self.samples = new
```

Функции создания и работы с экземплярами классов

```
def create_collection():
    choice = input("1 — вручную, 2 — случайно: ")

    samples = []

    if choice == "1":
        n = int(input("Количество образцов: "))

        for _ in range(n):
            while True:
                try:
                    gc = int(input("GC: "))
                    tm = int(input("Tm: "))
                    length = int(input("Length: "))
                    samples.append(GeneticSample(gc, tm, length))
                    break
                except:
                    print("Ошибка ввода")

    else:
        n = int(input("Количество образцов: "))
```

```
        for _ in range(n):
            gc = random.randint(25, 75)
            tm = random.randint(60, 95)
            length = random.randint(500, 5000)
            samples.append(GeneticSample(gc, tm, length))

    return SampleCollection(samples)


def show(collection):
    for s in collection:
        print(s)


def menu():
    col = create_collection()

    while True:
        print()
        print("1 Показать")
        print("2 Удалить GC-бедные")
        print("3 Уникальные")
        print("4 Самый длинный фрагмент")
        print("5 Средние параметры")
        print("6 Сдвиг (вектор)")
        print("0 Выход")

        cmd = input("Выбор: ")

        if cmd == "1":
            show(col)

        elif cmd == "2":
            col.remove_gc_poor()

        elif cmd == "3":
            col.unique_samples()

        elif cmd == "4":
            print(col.longest_fragment())

        elif cmd == "5":
            print(col.collection_center())

        elif cmd == "6":
            gc = int(input("GC shift: "))
            tm = int(input("Tm shift: "))
            length = int(input("Len shift: "))
            col.apply_vector((gc, tm, length))

        elif cmd == "0":
            break


menu()
```

Основная программа

```
import random

class GeneticSample:
    def __init__(self, gc, tm, length):
        if not (20 <= gc <= 80):
            raise ValueError
        if not (50 <= tm <= 100):
            raise ValueError
        if not (100 <= length <= 10000):
            raise ValueError

        self.gc = gc
        self.tm = tm
        self.length = length

    def __add__(self, other):
        if isinstance(other, GeneticSample):
```

```
            return GeneticSample(
                self.gc + other.gc,
                self.tm + other.tm,
                self.length + other.length
            )
        elif isinstance(other, (tuple, list)) and len(other) == 3:
            return GeneticSample(
                self.gc + other[0],
                self.tm + other[1],
                self.length + other[2]
            )
        else:
            raise TypeError

    def __sub__(self, other):
        if isinstance(other, GeneticSample):
            return GeneticSample(
                self.gc - other.gc,
                self.tm - other.tm,
                self.length - other.length
            )
        elif isinstance(other, (tuple, list)) and len(other) == 3:
            return GeneticSample(
                self.gc - other[0],
                self.tm - other[1],
                self.length - other[2]
            )
        else:
            raise TypeError

    def __mul__(self, k):
        if not isinstance(k, (int, float)):
            raise TypeError
        return GeneticSample(
            int(self.gc * k),
            int(self.tm * k),
            int(self.length * k)
        )

    def __eq__(self, other):
        return (self.gc, self.tm, self.length) == (other.gc, other.tm, other.length)

    def __hash__(self):
        return hash((self.gc, self.tm, self.length))

    def __bool__(self):
        return self.gc > 50

    def __str__(self):
        return f"GC: {self.gc}%, Tm: {self.tm}°C, Len: {self.length}"


class SampleCollection:
    def __init__(self, samples=None):
        self.samples = samples if samples else []

    def __getitem__(self, index):
        return self.samples[index]

    def __len__(self):
        return len(self.samples)

    def __iter__(self):
        return iter(self.samples)

    def remove_gc_poor(self):
        self.samples = [s for s in self.samples if s.gc > 50]

    def unique_samples(self):
        self.samples = list(set(self.samples))

    def longest_fragment(self):
        if not self.samples:
            return None
        return max(self.samples, key=lambda x: x.length)

    def collection_center(self):
```

```
        if not self.samples:
            return (0, 0, 0)

        gc = sum(s.gc for s in self.samples) / len(self.samples)
        tm = sum(s.tm for s in self.samples) / len(self.samples)
        length = sum(s.length for s in self.samples) / len(self.samples)

        return (gc, tm, length)

    def apply_vector(self, vector):
        new = []
        for s in self.samples:
            new.append(s + vector)
        self.samples = new
def create_collection():
    choice = input("1 — вручную, 2 — случайно: ")

    samples = []

    if choice == "1":
        n = int(input("Количество образцов: "))

        for _ in range(n):
            while True:
                try:
                    gc = int(input("GC: "))
                    tm = int(input("Tm: "))
                    length = int(input("Length: "))
                    samples.append(GeneticSample(gc, tm, length))
                    break
                except:
                    print("Ошибка ввода")

    else:
        n = int(input("Количество образцов: "))

        for _ in range(n):
            gc = random.randint(25, 75)
            tm = random.randint(60, 95)
            length = random.randint(500, 5000)
            samples.append(GeneticSample(gc, tm, length))

    return SampleCollection(samples)


def show(collection):
    for s in collection:
        print(s)


def menu():
    col = create_collection()

    while True:
        print()
        print("1 Показать")
        print("2 Удалить GC-бедные")
        print("3 Уникальные")
        print("4 Самый длинный фрагмент")
        print("5 Средние параметры")
        print("6 Сдвиг (вектор)")
        print("0 Выход")

        cmd = input("Выбор: ")

        if cmd == "1":
            show(col)

        elif cmd == "2":
            col.remove_gc_poor()

        elif cmd == "3":
            col.unique_samples()

        elif cmd == "4":
            print(col.longest_fragment())

        elif cmd == "5":
```

```
            print(col.collection_center())

        elif cmd == "6":
            gc = int(input("GC shift: "))
            tm = int(input("Tm shift: "))
            length = int(input("Len shift: "))
            col.apply_vector((gc, tm, length))

        elif cmd == "0":
            break


menu()
```

Задание 3

📝 Задание 3. Индексация, изменение данных и итераторы

🎯 Формулировка задания 3:

* 📊 «Матрица экспрессии генов»

📌 Суть задания Создать программу для работы с таблицей экспрессии генов в различных условиях. Таблица представляет собой сетку генов и условий, каждая ячейка — уровень экспрессии (логарифмическое значение). Требуется реализовать безопасный доступ, проверку типов и операции над таблицей.

🔒 Валидация данных через классы Класс ExpressionRow — строка таблицы экспрессии: @property row — список чисел; элементы должны быть числами с плавающей точкой (-10..+10 лог.ед.); Класс ExpressionMatrix — таблица экспрессии генов: @property rows — список объектов ExpressionRow; все строкы одной длины; все элементы числа; Ошибки типа: TypeError или ValueError. ⚙️ Функциональность Доступ и изменение элемента: matrix[i][j] (уровень экспрессии гена); Удаление столбца: del matrix[:, j] (исключение условия эксперимента); Сложение двух таблиц одинакового размера: matrix1 + matrix2 (суммирование данных); Умножение таблицы на число: matrix * k (шкалирование); Метод average_expression() — средний уровень экспрессии генов. 🧑‍💻 Ввод данных 1 — Ввести таблицу вручную 2 — Случайная генерация Вручную: ввод размеров + значения уровней экспрессии; Генерация: числа из диапазона [-5, 5]. 🧪 Операции в программе Показать таблицу экспрессии генов Изменить уровень экспрессии гена Удалить условие эксперимента (столбец) Умножить на коэффициент (шкалирование) Сложить с другой таблицей экспрессии Посчитать средний уровень экспрессии *

Описание классов

```
class ExpressionRow:
    def __init__(self, values):
        self.row = values

    @property
    def row(self):
        return self._row

    @row.setter
    def row(self, values):
        if not all(isinstance(x, (int, float)) for x in values):
            raise TypeError
        if any(not (-10 <= x <= 10) for x in values):
            raise ValueError
        self._row = [float(x) for x in values]

    def __len__(self):
        return len(self._row)

    def __getitem__(self, idx):
        return self._row[idx]

    def __setitem__(self, idx, value):
        if not (-10 <= value <= 10):
            raise ValueError
        self._row[idx] = float(value)
```

```
class ExpressionMatrix:
    def __init__(self, rows):
        self.rows = rows

    @property
    def rows(self):
        return self._rows

    @rows.setter
    def rows(self, value):
        if not all(isinstance(r, ExpressionRow) for r in value):
            raise TypeError
        if len(value) > 0:
            ln = len(value[0])
            if any(len(r) != ln for r in value):
                raise ValueError
        self._rows = value

    def __getitem__(self, idx):
        return self._rows[idx]

    def __delitem__(self, key):
        if isinstance(key, tuple) and key[0] == slice(None):
            col_idx = key[1]
            for r in self._rows:
                r._row.pop(col_idx)
        else:
            self._rows.pop(key)

    def __add__(self, other):
        if len(self.rows) != len(other.rows) or len(self.rows[0]) != len(other.rows[0]):
            raise ValueError
        res = []
        for i in range(len(self.rows)):
            new_vals = [self.rows[i][j] + other.rows[i][j] for j in range(len(self.rows[0]))]
            res.append(ExpressionRow(new_vals))
        return ExpressionMatrix(res)

    def __mul__(self, k):
        res = []
        for r in self._rows:
            new_vals = [x * k for x in r.row]
            res.append(ExpressionRow(new_vals))
        return ExpressionMatrix(res)

    def average_expression(self):
        total = sum(sum(r.row) for r in self._rows)
        count = len(self._rows) * len(self._rows[0])
        return total / count if count > 0 else 0

    def __str__(self):
        return "\n".join("\t".join(f"{x:.2f}" for x in r.row) for r in self._rows)
```

Функции создания и работы с экземплярами классов

```
def create_matrix():
    choice = input("1 — вручную, 2 — случайно: ")
    r = int(input("Количество генов: "))
    c = int(input("Количество условий: "))
    data = []
    if choice == "1":
        for i in range(r):
            vals = list(map(float, input(f"Ген {i+1}: ").split()))
            data.append(ExpressionRow(vals))
    else:
        for _ in range(r):
            vals = [random.uniform(-5, 5) for _ in range(c)]
            data.append(ExpressionRow(vals))
    return ExpressionMatrix(data)

def menu():
    matrix = create_matrix()
    while True:
        print("\n1 Показать таблицу\n2 Изменить значение\n3 Удалить условие (столбец)\n4 Умножить на число\n5 Сложит
```

```
        cmd = input("Выбор: ")
        if cmd == "1":
            print(matrix)
        elif cmd == "2":
            i = int(input("Индекс гена: "))
            j = int(input("Индекс условия: "))
            v = float(input("Новое значение: "))
            matrix[i][j] = v
        elif cmd == "3":
            j = int(input("Индекс столбца для удаления: "))
            del matrix[:, j]
        elif cmd == "4":
            k = float(input("Коэффициент: "))
            matrix = matrix * k
        elif cmd == "5":
            print("Создание второй матрицы...")
            other = create_matrix()
            matrix = matrix + other
        elif cmd == "6":
            print(f"Среднее: {matrix.average_expression():.4f}")
        elif cmd == "0":
            break
```

Основная программа

```
import random

class ExpressionRow:
    def __init__(self, values):
        self.row = values

    @property
    def row(self):
        return self._row

    @row.setter
    def row(self, values):
        if not all(isinstance(x, (int, float)) for x in values):
            raise TypeError
        if any(not (-10 <= x <= 10) for x in values):
            raise ValueError
        self._row = [float(x) for x in values]

    def __len__(self):
        return len(self._row)

    def __getitem__(self, idx):
        return self._row[idx]

    def __setitem__(self, idx, value):
        if not (-10 <= value <= 10):
            raise ValueError
        self._row[idx] = float(value)

class ExpressionMatrix:
    def __init__(self, rows):
        self.rows = rows

    @property
    def rows(self):
        return self._rows

    @rows.setter
    def rows(self, value):
        if not all(isinstance(r, ExpressionRow) for r in value):
            raise TypeError
        if len(value) > 0:
            ln = len(value[0])
            if any(len(r) != ln for r in value):
                raise ValueError
        self._rows = value

    def __getitem__(self, idx):
        return self._rows[idx]
```

```
    def __delitem__(self, key):
        if isinstance(key, tuple) and key[0] == slice(None):
            col_idx = key[1]
            for r in self._rows:
                r._row.pop(col_idx)
        else:
            self._rows.pop(key)

    def __add__(self, other):
        if len(self.rows) != len(other.rows) or len(self.rows[0]) != len(other.rows[0]):
            raise ValueError
        res = []
        for i in range(len(self.rows)):
            new_vals = [self.rows[i][j] + other.rows[i][j] for j in range(len(self.rows[0]))]
            res.append(ExpressionRow(new_vals))
        return ExpressionMatrix(res)

    def __mul__(self, k):
        res = []
        for r in self._rows:
            new_vals = [x * k for x in r.row]
            res.append(ExpressionRow(new_vals))
        return ExpressionMatrix(res)

    def average_expression(self):
        total = sum(sum(r.row) for r in self._rows)
        count = len(self._rows) * len(self._rows[0])
        return total / count if count > 0 else 0

    def __str__(self):
        return "\n".join("\t".join(f"{x:.2f}" for x in r.row) for r in self._rows)

def create_matrix():
    choice = input("1 — вручную, 2 — случайно: ")
    r = int(input("Количество генов: "))
    c = int(input("Количество условий: "))
    data = []
    if choice == "1":
        for i in range(r):
            vals = list(map(float, input(f"Ген {i+1}: ").split()))
            data.append(ExpressionRow(vals))
    else:
        for _ in range(r):
            vals = [random.uniform(-5, 5) for _ in range(c)]
            data.append(ExpressionRow(vals))
    return ExpressionMatrix(data)

def menu():
    matrix = create_matrix()
    while True:
        print("\n1 Показать таблицу\n2 Изменить значение\n3 Удалить условие (столбец)\n4 Умножить на число\n5 Сложить
        cmd = input("Выбор: ")
        if cmd == "1":
            print(matrix)
        elif cmd == "2":
            i = int(input("Индекс гена: "))
            j = int(input("Индекс условия: "))
            v = float(input("Новое значение: "))
            matrix[i][j] = v
        elif cmd == "3":
            j = int(input("Индекс столбца для удаления: "))
            del matrix[:, j]
        elif cmd == "4":
            k = float(input("Коэффициент: "))
            matrix = matrix * k
        elif cmd == "5":
            print("Создание второй матрицы...")
            other = create_matrix()
            matrix = matrix + other
        elif cmd == "6":
            print(f"Среднее: {matrix.average_expression():.4f}")
        elif cmd == "0":
            break

if __name__ == "__main__":
    menu()
```